Introduction to Computer Systems

Spring 2026

Lecture 2
Processors and Assembly Code

Instructors: Wei Xu, Mingyu Gao

Teaching Assistants: Weitao Wan, Tingrui Zhang, Ruitao Liu, Shuo Chen

Contents

1 Introduction **3**

2 High-Level Models of Computers and Processors **3**

2.1 The computer as an input–process–output system 3

2.2 Von Neumann architecture and the stored-program idea 4

2.3 Logical components versus physical systems 5

2.4 From source code to executable instructions 6

3 Processor Hardware Structures: From Correctness to Performance **7**

3.1 A minimal processor model 7

3.2 Registers as a fundamental optimization 8

3.3 The fetch–decode–execute view of instruction execution 8

3.4 The simple serial processor 9

3.5 Pipelining: improving throughput by overlap 9

3.6 Superscalar execution and dynamic independence 11

3.7 Multi-core processors: parallelism across cores 11

3.8 GPU and accelerator-oriented parallelism 12

4 Instruction Set Architecture and RV32I Assembly **14**

4.1 RISC and CISC as ISA design styles 14

4.2 Why RISC-V is a good teaching ISA 15

4.3 Architectural state in RV32I 15

4.4 Instruction syntax and operand conventions 16

4.5 Arithmetic and logical instructions 17

4.6 Load/store design and displacement addressing 17

4.7 Arrays, pointers, and address calculation 18

4.8 Control flow: branches, jumps, and structured programming 19

5 ISA Support for Parallelism and Synchronization **21**

5.1 SIMD: single instruction, multiple data 21

5.2 Case study: matrix multiplication and vectorization 21

5.3 Shared memory, data races, and atomicity 23

5.4 Atomic instructions and synchronization primitives 24

6 Memory Layout, Stack Management, and Procedure Calls **25**

6.1 The memory layout of a process 25

6.2 Why a stack is needed 27

6.3 Stack frames and nested procedure calls 28

6.4 Register interference across procedure calls 29

6.5 Procedure calls as control transfer plus state management 29

6.6 Programming implications and common pitfalls 30

7 Summary **30**

8 Quizzes & Exercises **31**

8.1 Concept checks drawn from the lecture 31

8.2 Short-answer quiz 31

8.3 Essay and discussion prompts 32

8.4 Additional practice exercises 32

1 Introduction

The central question of this lecture is deceptively simple: *how does a program become computation?* People writing C or C++ usually see variables, loops, functions, and data structures. A processor, however, does not execute "variables" or "functions" as such. It executes instructions, and those instructions operate over an explicit architectural state consisting of a program counter, registers, and memory. Understanding that transformation—from high-level intent to low-level machine action—is one of the foundational ideas of computer systems.

This lecture therefore sits at an important conceptual junction. It begins with abstract models of a computer system, then moves inward to processor organization, and then moves back outward to the software-visible interface known as the instruction set architecture (ISA). Along the way, it explains why modern processors are built the way they are: why registers exist, why pipelining increases throughput, why multi-core systems require synchronization, and why the stack is the natural mechanism for supporting nested procedure calls.

Although the lecture uses RISC-V as its running example, the underlying ideas are much broader. Most modern architectures, whether x86-64, ARM, or RISC-V, solve the same fundamental problems: representing state, transporting data, sequencing control flow, exploiting parallelism, and managing temporary storage. RISC-V is especially useful pedagogically because it exposes these ideas with relatively little historical baggage. In this note, the 32-bit integer subset RV32I will be used to illustrate the core concepts.

Definition

An **instruction set architecture (ISA)** is the software-visible specification of a processor. It defines the architectural state (for example, registers and memory behavior), the available instructions, and the meaning of those instructions. It is the main interface between software and hardware.

Two pedagogical goals guide the rest of this note. First, every hardware mechanism should be understood in terms of *why it exists*. Second, every assembly-language construct should be understood not as arbitrary syntax, but as a precise reflection of hardware capabilities and constraints. The most durable understanding of computer systems comes from seeing these two directions at once.

2 High-Level Models of Computers and Processors

2.1 The computer as an input–process–output system

At the highest level of abstraction, a computer system receives input, performs some computation, and produces output. The input may come from a human user, a network connection, a sensor, a disk, or another component in a distributed system. The output may similarly be displayed, transmitted, stored, or used to control another physical device. This view is intentionally abstract, but it is a useful starting point because it emphasizes that computation is always embedded within an environment.

Even such a simple description already suggests the major components a computer must contain:

- **Input/output devices**, to exchange information with the external world.
- **A compute unit**, to transform input data into output data.
- **Memory**, to hold both persistent and temporary information.
- **A control mechanism**, to determine which operations are performed and in what order.

The control component matters because general-purpose computers do not perform just one fixed computation. The same hardware may be used to render graphics, sort records, run a web browser, train a machine-learning model, or process a spreadsheet. That flexibility requires a programmable description of behavior: the *program*.

2.2 Von Neumann architecture and the stored-program idea

The classic stored-program computer is usually described through the Von Neumann architecture, also called the Princeton architecture. Its core insight is that a computer's memory stores *both* data and the program that operates on that data. The processor is then responsible for fetching instructions from memory, interpreting them, and updating the machine state accordingly.

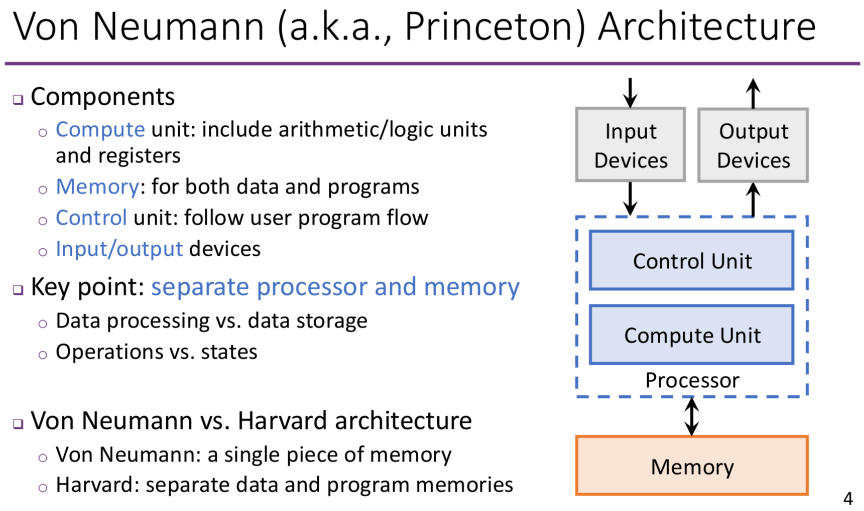


Figure 1: Stored-program organization of a Von Neumann system, extracted from slide 4. The figure emphasizes the logical separation between the processor and memory, with the processor containing control and compute logic while the memory holds both program instructions and data.

This model is significant for at least three reasons.

First, it separates *data processing* from *data storage*. The processor performs operations; the memory holds state. At an even more abstract level, one may say that the processor implements *state transitions*: it reads the current state, applies an operation, and produces a new state.

Second, the model treats the program itself as data in memory. This stored-program idea is historically profound because it makes general-purpose computing possible. A single machine can perform many different tasks simply by loading a different instruction sequence.

Third, the model creates a natural interface between hardware and software. Software provides an instruction sequence; hardware executes it. Everything else in systems design can be seen as elaborating, optimizing, or constraining this basic contract.

A frequently mentioned alternative is the **Harvard architecture**, which uses separate memories for instructions and data. Logically, the difference is straightforward: the Von Neumann model uses one unified memory, while the Harvard model separates the two classes of information. In practice, modern processors often blur the distinction. For example, a machine may preserve a unified logical address space while internally using separate instruction and data caches. Thus, the most important lesson is not the taxonomy itself, but the broader point that multiple physical implementations may realize the same logical abstraction.

Deep Dive

The distinction between *architecture* and *implementation* appears very early here. A programmer reasons about the existence of memory, instructions, and state transitions. The hardware designer worries about whether instruction and data paths are physically separate, how the memory is packaged, and how fast each access can occur. The former belongs to architecture; the latter belongs to microarchitecture.

2.3 Logical components versus physical systems

The elegance of the abstract model should not mislead us into thinking that real machines are physically simple. A modern phone, laptop, or server is a heterogeneous collection of subsystems with different performance, energy, and specialization goals.

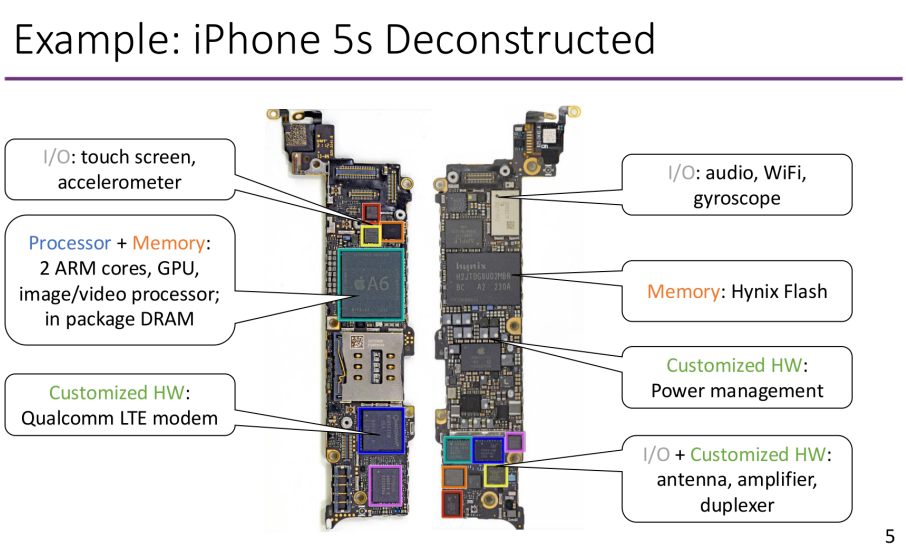


Figure 2: Deconstructed iPhone 5s board layout from slide 5. The annotation illustrates that a real computer system includes processors, memories, communication circuitry, audio and sensor interfaces, and multiple pieces of customized hardware. Logical abstractions such as "processor" and "memory" may map to physically packaged or distributed components in several ways.

The iPhone example is pedagogically useful because it shows that even a compact consumer device already contains not just a CPU, but also GPU logic, memory packages, flash storage, modems, power-management circuits, radio-frequency components, and sensor interfaces. Some of these are general-purpose, while others are highly specialized. This observation motivates an important systems principle:

> A general-purpose solution is often functionally sufficient, but a specialized solution may be much more efficient.

In other words, a CPU can in principle carry out many tasks that are also performed by a GPU, a modem, or a neural-network accelerator. Yet doing everything on the CPU would often be too slow or too energy-expensive. Specialized hardware exists not because general-purpose processing is impossible, but because specialization can reduce overhead and exploit domain-specific structure.

2.4 From source code to executable instructions

People usually write programs in a high-level language. A processor does not directly understand C, C++, or Python source code. Instead, a translation pipeline progressively lowers the program into machine-understandable form.

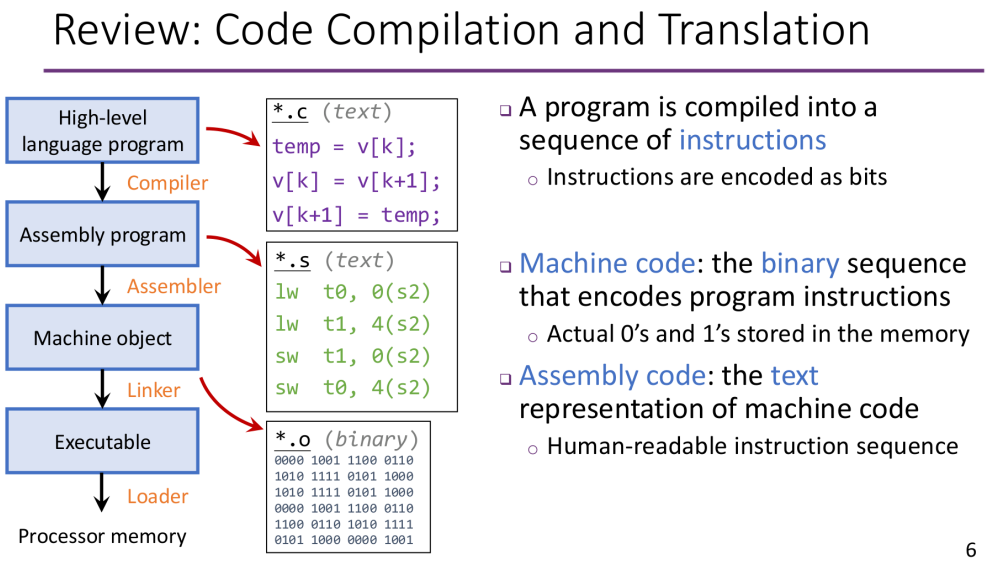


Figure 3: Compilation and translation path from source code to machine code, extracted from slide 6. A compiler lowers a high-level program to assembly, an assembler converts that representation into machine code, a linker combines program fragments into an executable, and a loader places the executable into memory for execution.

The key stages are as follows:

- A **compiler** translates a high-level language program into a lower-level representation, often assembly code.
- An **assembler** converts assembly language into machine code, that is, the binary bit patterns that encode instructions.
- A **linker** combines object files and external library references into an executable program.
- A **loader** places the executable into memory and prepares the machine state so execution can begin.

The distinction between **machine code** and **assembly code** is especially important. Machine code is what the processor actually fetches from memory: numeric bit patterns. Assembly language is a human-readable textual notation for those bit patterns. The two are different representations of the same low-level program.

This leads directly to the notion of ISA as the hardware–software interface. Software can be compiled for a given ISA without knowing the transistor-level details of the chip. Meanwhile, hardware designers can change pipelines, cache structures, issue widths, and clock frequencies without requiring all software to be rewritten, so long as they preserve the ISA contract.

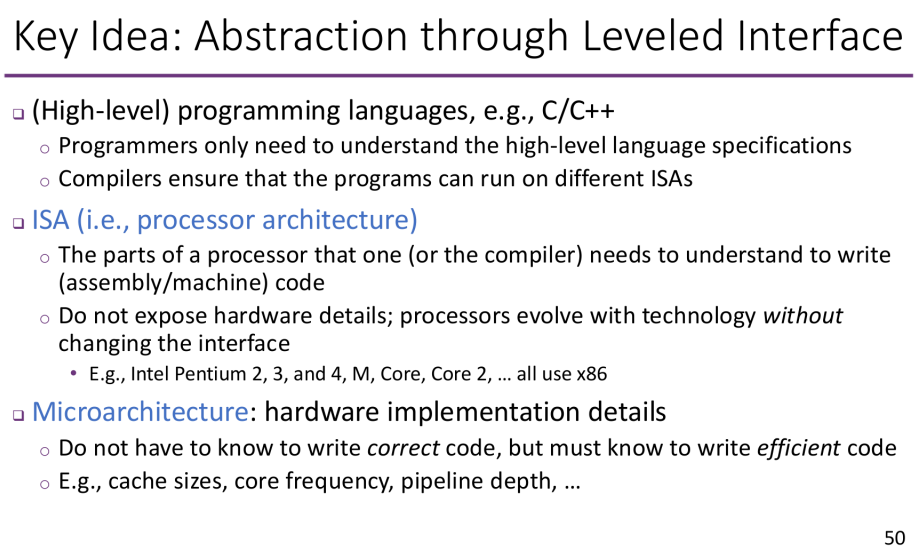


Figure 4: Abstraction through a leveled interface, extracted from slide 50. The slide distinguishes between high-level languages, ISA, and microarchitecture. This hierarchy is one of the central organizing ideas of computer systems.

3 Processor Hardware Structures: From Correctness to Performance

3.1 A minimal processor model

A first processor model contains four essential ingredients: a program counter (PC), memory, an arithmetic/logic unit (ALU), and a controller. The PC identifies the address of the next instruction to fetch. Memory contains both code and data. The ALU performs arithmetic and logical operations. The controller interprets instructions and orchestrates the movement of information.

This model is functionally complete, but not yet efficient. If every piece of data must be read from memory before being used and written back immediately afterward, the processor spends much of its time waiting on memory traffic. That is undesirable because memory is relatively large but relatively slow.

3.2 Registers as a fundamental optimization

To reduce this bottleneck, processors include **registers**: a small set of very fast storage locations located directly inside the processor. Registers hold values that are actively being manipulated. They are much closer to the ALU than main memory is, so accessing them is far faster.

The design trade-off is a classic one: *smaller is faster*. A processor can afford only a limited number of registers because speed, area, and complexity all constrain how large this storage can be. Consequently, the compiler or assembly programmer must decide which values deserve to reside in registers at each point in time.

This decision has major performance implications. Frequently used variables, loop indices, address pointers, and temporary results are especially attractive register candidates. Values that are less frequently used, or too numerous to fit, must remain in memory.

3.3 The fetch–decode–execute view of instruction execution

At a conceptual level, instruction execution proceeds through a repeating cycle:

1. Use the PC to fetch an instruction from memory.
2. Decode that instruction and determine what operations it requires.
3. Read source operands, perform computation, and/or access memory.
4. Write the result back if necessary.
5. Update the PC to identify the next instruction.

A common hardware decomposition of this cycle is the classic five-stage pipeline: *Fetch*, *Decode*, *Execute*, *Load/Store*, and *Write Back*.

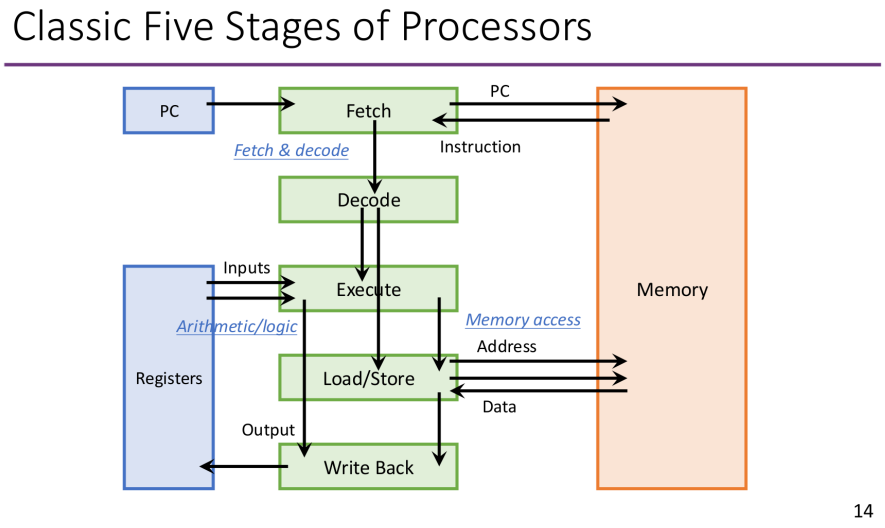


Figure 5: Classic five-stage processor organization, extracted from slide 14. The figure shows the architectural flow from PC-based instruction fetch, to register operand decoding, to arithmetic execution, optional memory access, and finally register write-back.

Algorithm 1: Abstract Instruction Execution Cycle

Input: Architectural state consisting of PC, registers, and memory
Output: Updated architectural state after executing one instruction

```
instr ← Memory[PC];
Decode instr and determine the required action;
Read any source operands from the register file;
Execute arithmetic/logic and compute any effective addresses;
if instr requires memory access then
    Perform the load or store operation;
if instr produces a register result then
    Write the result into the destination register;
Update PC to the sequential next instruction or to a branch/jump target;
return updated state;
```

Not every instruction uses every stage. An arithmetic instruction such as `add` does not need a memory access. A store instruction writes memory but typically does not write a result register. Yet the hardware must still implement the full union of these capabilities so that the complete ISA can be supported.

3.4 The simple serial processor

The simplest realization of the five-stage model is strictly sequential. One instruction passes through all necessary stages, and only after it completes does the next instruction begin. Functionally, this is straightforward and easy to reason about. The latency of one instruction is the sum of the times of its stages, and the total execution time of a program is the sum over all instructions.

The weakness is equally clear: there is no overlap. While one instruction is being decoded, the fetch hardware sits idle. While another is performing an ALU operation, the memory-access hardware may have nothing to do. Throughput is therefore much lower than the available hardware structure could potentially support.

3.5 Pipelining: improving throughput by overlap

Pipelining addresses this inefficiency by allowing different instructions to occupy different stages simultaneously. Once the first instruction advances from Fetch to Decode, the Fetch stage can begin working on the next instruction. The hardware stages are thus reused in an overlapped fashion.

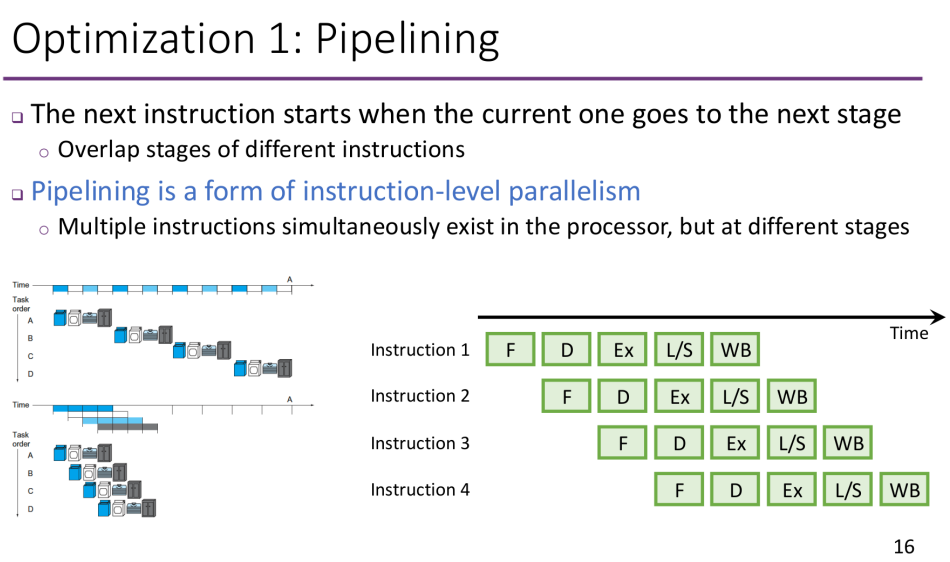


Figure 6: Pipelined execution from slide 16. Different instructions occupy different hardware stages at the same time, increasing throughput without changing the architectural meaning of the program.

The most important conceptual distinction is this: **pipelining improves throughput more than it improves single-instruction latency**. A five-stage pipeline does not make one instruction conceptually require fewer steps; instead, it allows many instructions to be "in flight" at once. In the idealized case, once the pipeline is full, the processor may complete roughly one instruction per cycle, even though each individual instruction still traverses multiple stages.

However, real pipelines are not ideal. Three classic hazard classes limit performance:

- **Structural hazards**: two operations need the same hardware resource at the same time.
- **Data hazards**: an instruction depends on a result that has not yet been produced by an earlier instruction.
- **Control hazards**: the processor does not yet know which instruction should come next because a branch outcome has not been resolved.

Data hazards motivate forwarding and stalling mechanisms. Control hazards motivate branch prediction and speculative execution in more advanced machines. Although the lecture does not require detailed hazard machinery, you should remember the big idea: overlap is powerful, but it creates coordination problems.

Deep Dive

The ISA does not say whether a processor is pipelined. A non-pipelined processor and a deeply pipelined processor may execute the same program and produce identical architectural results. Pipelining is therefore a microarchitectural optimization, not an architectural requirement.

3.6 Superscalar execution and dynamic independence

Pipelining overlaps *different stages* of different instructions. A further step is to execute multiple independent instructions at the *same stage* in the same cycle. This is the idea of a **superscalar** processor.

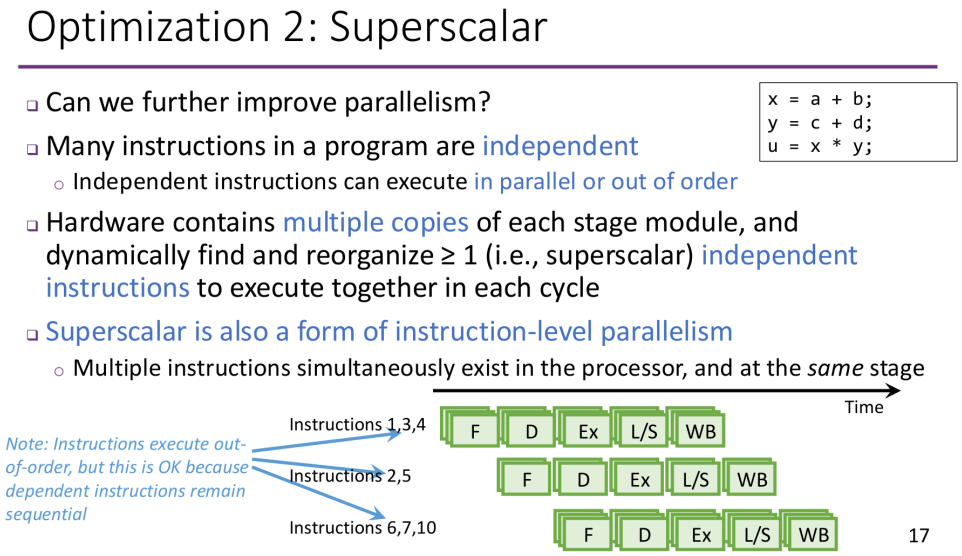


Figure 7: Superscalar execution from slide 17. The processor contains multiple copies of certain execution resources and dynamically issues more than one independent instruction in a cycle. Dependency constraints still enforce correctness.

The key observation is that many instructions in a program are independent. If instruction u depends on the values produced by instructions x and y, then u must wait. But if two instructions use unrelated inputs and produce unrelated outputs, the processor may execute them simultaneously or even reorder them. Such reordering is safe so long as true data dependencies are preserved.

Modern superscalar processors therefore contain multiple functional units and hardware logic to detect dependencies. More advanced designs also support **out-of-order execution**: instructions are allowed to execute as soon as their operands are ready, even if earlier instructions are still waiting on a long-latency event such as a cache miss.

This is one of the most important places where you should appreciate the difference between *program order* and *execution order*. The ISA defines the effect of the program as if instructions were executed according to the architectural rules. The microarchitecture is free to reorder internal work so long as the final visible behavior is correct.

3.7 Multi-core processors: parallelism across cores

A single core can exploit instruction-level parallelism, but the amount of independence available within one instruction stream is limited. Modern processors therefore also exploit **thread-level** or **task-level** parallelism through multiple cores.

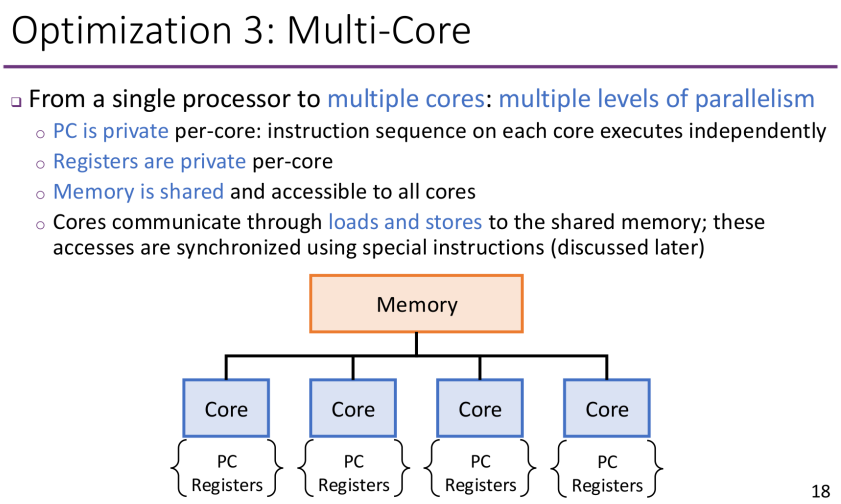


Figure 8: Multi-core organization from slide 18. Each core has its own program counter and registers, while memory is shared. Communication between cores therefore occurs through the shared memory system.

Each core typically has its own private PC and register state, meaning that each core can execute a different instruction stream. The shared memory becomes the medium through which cores communicate. One core may write data that another later reads.

This design immediately raises a new challenge: if multiple cores access the same memory location, what guarantees keep the result sensible? That question will lead us later to atomic operations and synchronization. For now, the main point is that multi-core parallelism differs from pipelining and superscalar execution because it involves distinct architectural contexts, not merely more aggressive execution of a single instruction stream.

3.8 GPU and accelerator-oriented parallelism

GPUs offer an even more throughput-oriented form of parallel hardware. They are built to process very large numbers of similar operations across large data sets, which makes them ideal for graphics and many machine-learning workloads.

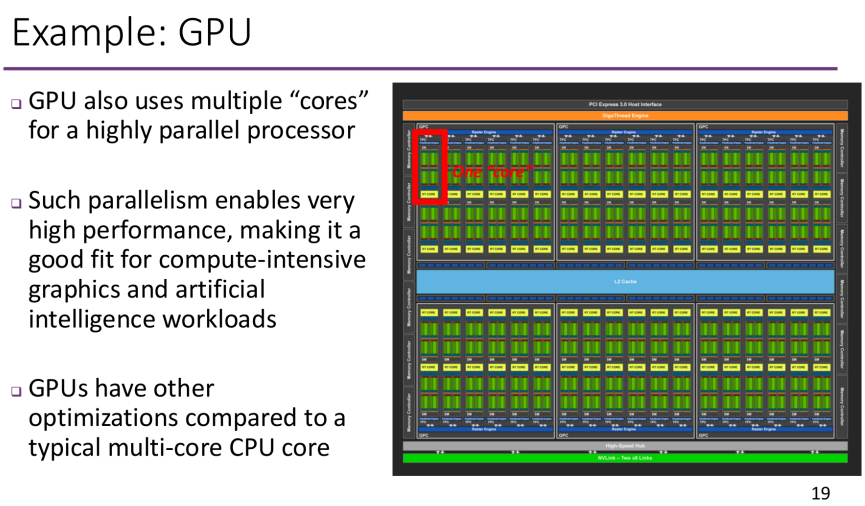


Figure 9: GPU example from slide 19. The figure conveys the idea that a GPU contains many repeating compute structures, providing far more parallel throughput than a typical CPU core.

A GPU is not simply "many CPUs glued together." It is designed with a different emphasis: very high throughput, many concurrent operations, and a willingness to devote less hardware budget per individual control flow path. CPUs typically optimize for low latency on complex, branch-heavy programs. GPUs typically optimize for massive throughput on highly regular, data-parallel workloads.

At the specialized end of the spectrum lie hardware accelerators such as tensor processors.

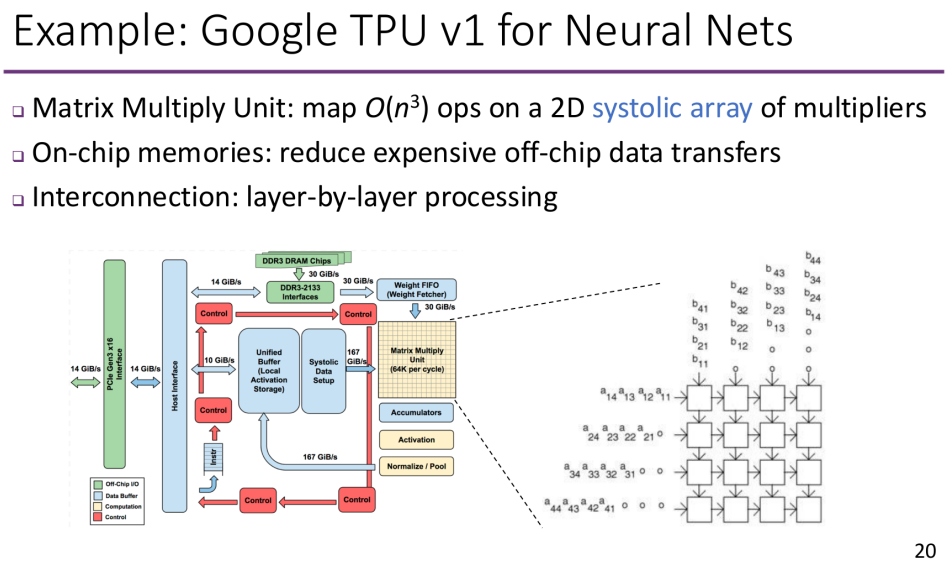


Figure 10: Google TPU v1 organization from slide 20. The matrix-multiplication systolic array, on-chip memories, and dataflow-centric organization illustrate how a specialized accelerator removes much of the general-purpose overhead present in CPUs.

A tensor processing unit (TPU) exemplifies domain specialization. Matrix multiplication, which is central to neural-network workloads, is mapped onto a large systolic array. On-chip memories reduce costly off-chip transfers, and the interconnection network is tuned for layer-by-layer

processing. The lesson is broad: the more structure a workload has, the more opportunity there is to create hardware that directly matches that structure.

Level of parallelism	**Typical mechanism**	**Main idea**
Data-level	SIMD/vector instructions	One instruction operates on several data elements at once.
Instruction-level	Pipelining, superscalar, out-of-order	One core overlaps or co-issues multiple instructions from a single instruction stream.
Thread/task-level	Multi-core processors	Different cores execute different control flows that communicate through memory.
Domain-specific	GPU and accelerators	Hardware structure is specialized for a narrow workload class to maximize throughput and efficiency.

Table 1: Multiple levels of parallelism that can coexist in a modern system.

4 Instruction Set Architecture and RV32I Assembly

4.1 RISC and CISC as ISA design styles

Two historically influential ISA styles are **RISC** (Reduced Instruction Set Computer) and **CISC** (Complex Instruction Set Computer). The distinction is sometimes overstated, but it is pedagogically useful.

Aspect	**RISC**	**CISC**
Instruction philosophy	Many simple instructions, each doing a relatively small amount of work	Richer instructions, each potentially combining multiple lower-level operations
Hardware implication	Simpler decoding and cleaner execution structures	More complex decoding and more irregular instruction behavior
Program implication	Usually more instructions per program	Usually fewer instructions per program
Examples	RISC-V, ARM, MIPS	x86, x86-64

Table 2: Pedagogical comparison between RISC and CISC. Real modern machines often blur the boundary.

Historically, CISC designs were attractive when memory was scarce and programmers worked closer to assembly. Rich instructions could make programs shorter. RISC gained momentum when compiler technology improved, pipeline-friendly simplicity became more valuable, and energy efficiency made complex decode logic more expensive to justify. Today, the line is blurred: modern x86 processors often translate complex instructions internally into simpler micro-operations that resemble RISC-style execution.

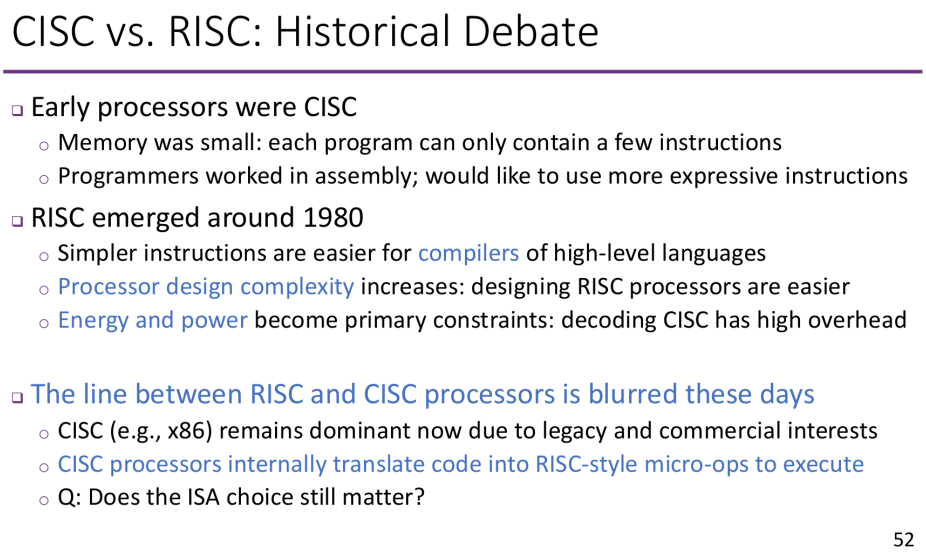


Figure 11: Historical perspective on the RISC–CISC debate from slide 52. The modern lesson is not that one side "won" absolutely, but that implementation trends have pushed even CISC processors toward simpler internal execution representations.

4.2 Why RISC-V is a good teaching ISA

RISC-V is the fifth generation of the RISC line originating at UC Berkeley. It is widely used in education because it is clean, regular, and open. Its simplicity exposes core architectural ideas with less historical irregularity than older, heavily evolved ISAs.

In this lecture, the relevant subset is RV32I: the 32-bit integer base ISA. The course focus is on *reading* assembly code and relating it back to C-level meaning. That direction—assembly to C—is pedagogically important because it reveals what the compiler must ultimately produce, how control flow is realized, and why data movement matters.

4.3 Architectural state in RV32I

The architectural state of RV32I contains three principal components:

- a 32-bit **program counter** (PC),
- thirty-two 32-bit **general-purpose registers** named `x0` through `x31`, and
- a **byte-addressed memory**.

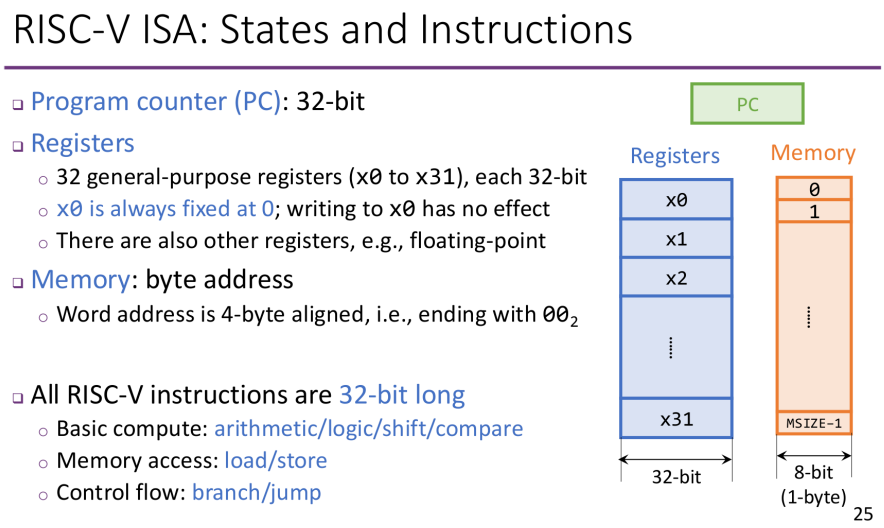


Figure 12: Architectural state in RV32I from slide 25. The ISA exposes a 32-bit program counter, a 32-register integer register file, and byte-addressed memory.

Several details are worth highlighting.

First, memory is *byte addressed*. Every byte has its own address. A 32-bit word therefore occupies four consecutive byte addresses. When one speaks of a "word-aligned" address in this setting, one means an address divisible by four.

Second, `x0` is special: it is permanently fixed to zero. Reads from `x0` always yield 0, and writes to `x0` have no effect.

Deep Dive

A hardwired zero register may at first seem wasteful, but it is extremely useful. It lets the ISA express common operations without needing extra instruction formats. For example, copying a register can be implemented as adding zero; zeroing a register can be implemented as XORing it with itself; comparing against zero becomes especially convenient. By dedicating one register to a constant, the ISA can remain smaller and cleaner.

Third, in RV32I all instructions are 32 bits long. Fixed-length instructions simplify fetching and decoding relative to highly variable-length encodings.

4.4 Instruction syntax and operand conventions

RV32I arithmetic and logical instructions commonly use forms such as

```
<op> rd, rs1, rs2      or      <op>i rd, rs1, imm
```

where `rd` is the destination register, `rs1` and `rs2` are source registers, and `imm` is an *immediate value*, that is, a constant encoded directly in the instruction.

The destination-first convention is important. In many introductory examples, we can instinctively read an instruction left-to-right as if it were ordinary infix mathematics, but assembly syntax is closer to an assignment form. For instance,

```
add x1, x2, x3
```

means that the value placed into `x1` is the sum of the current values in `x2` and `x3`. Conceptually, this corresponds to

```
x1 = x2 + x3
```

not to some symmetric three-way relation.

A useful note from the lecture is that not every seemingly reasonable mnemonic is necessarily a real primitive instruction. For example, a notation such as `subi` may be treated as a pseudo-instruction and implemented through `addi` with a negative immediate. This again reflects the RISC design philosophy: fewer fundamental instruction types, with other conveniences layered on top.

4.5 Arithmetic and logical instructions

The basic compute instructions include addition, subtraction, logical operations such as AND and XOR, shifts, and comparisons. A simple but revealing example is:

Listing 1: Elementary arithmetic and logical examples in RV32I.

```
add x1, x2, x3
addi x1, x1, -1
xor x1, x1, x1
```

The final instruction sets `x1` to zero, because any bit pattern XORed with itself yields all zeros. This is a good example of how a compact ISA leverages a small set of primitives to realize several common idioms.

Complex expressions are decomposed into simple steps. For example, consider the C statement

```
a = (b ^ c) + d - e;
```

Assuming `a`, `b`, `c`, `d`, and `e` reside in `x1` through `x5`, one possible translation is:

Listing 2: Breaking a compound expression into simple RISC-style instructions.

```
xor x28, x2, x3      # t0 = b ^ c
add x29, x28, x4     # t1 = t0 + d
sub x1, x29, x5      # a  = t1 - e
```

An instructive optimization question is how many temporary registers are *really* necessary. Since `x1` is the final destination for `a`, the computation may overwrite `x1` along the way if the program no longer needs its old value. Thus, zero extra temporaries may suffice in some cases. This is a small example of a big compiler concern: **register allocation**. Registers are precious, and careful reuse is essential.

4.6 Load/store design and displacement addressing

A hallmark of many RISC ISAs is that only load and store instructions access memory. Arithmetic instructions operate on register values only. This simplifies the hardware and regularizes the pipeline.

The central memory-transfer instructions in the lecture are:

`lw rd, offset(rs1)` and `sw rs2, offset(rs1)`

The effective address is computed by adding the signed immediate offset to the value stored in the base register `rs1`:

$$\text{effective address} = \text{register value} + \text{offset}.$$

Because memory is byte-addressed, this effective address is a byte address.

Definition

Displacement addressing (also called base-plus-offset addressing) is an addressing mode in which a memory address is computed by adding a constant offset to a base address stored in a register.

The following example implements

`*a = b + c[2];`

for 32-bit integers and pointers:

Listing 3: Load/store example with displacement addressing.

```
lw  x28, 8(x3)       # load c[2]
add x28, x2, x28     # b + c[2]
sw  x28, 0(x1)       # store result to *a
```

The offset is 8 rather than 2 because `c[2]` is two *elements* past the base, and each 32-bit integer occupies four bytes. This is a recurring source of confusion: array indexing is expressed in elements in C, but effective addresses are computed in bytes in the ISA.

Deep Dive

Why force memory access to occur only through loads and stores? The hardware reason is clarity. If every arithmetic instruction could directly fetch arbitrary operands from memory and write results back, the decode logic, execution structure, and addressing machinery would become more complicated. The software reason is that modern compilers are very good at decomposing high-level operations into explicit load, compute, and store steps, so much of the complexity can be shifted from hardware into compilation.

4.7 Arrays, pointers, and address calculation

Displacement addressing is especially natural for arrays and structure fields. If the array base address is stored in a register, then any element at a fixed index can be reached by embedding the corresponding byte offset directly into the instruction.

For example, if an integer array begins at address `0x1000`, then the addresses of its first few elements are:

`&a[0] = 0x1000`, `&a[1] = 0x1004`, `&a[2] = 0x1008`, and so on.

Thus, the expression

```
int x = a[7];
```

can be implemented as

```
lw x28, 28(x1)
```

if `x1` stores the base address of `a`.

When the index is a variable rather than a constant, the offset can no longer be encoded directly into the instruction. Instead, the processor must compute the address explicitly.

Listing 4: Computing the address of a variable-indexed array element.

```
slli x28, x1, 2      # x28 = 4 * i
add  x29, x2, x28    # x29 = a + 4*i = &a[i]
sw   x3, 0(x29)      # a[i] = x
addi x1, x1, 1       # i++
```

The instruction `slli` means "shift logical left by immediate." Shifting left by 2 bits multiplies a value by $2^2 = 4$, which is precisely the element size for 32-bit integers. For arrays of larger element types, the scaling factor would change accordingly. This illustrates an important interface between language semantics and ISA semantics: the compiler must know the element type in order to generate the correct byte-level address arithmetic.

4.8 Control flow: branches, jumps, and structured programming

By default, instructions execute sequentially. The PC advances from one instruction to the next in order. But high-level programs rely heavily on control-flow constructs such as `if`, `else`, `while`, `for`, and function calls. These require the ISA to provide ways to change the PC.

There are two main categories:

- **Conditional branches**, such as `beq` and `bne`, which change control flow only if some condition holds.
- **Unconditional jumps**, such as `j`, which always redirect execution.

The branch target is usually written as a *label*, which names a position in the instruction stream.

Example of if-else

```
if (a == d)
    a = a + d;
if (i == j)
    a = b + c;
else
    a = b - c;
a++;
```

```
# a, b, c, d in x1 to x4
# i, j in x5, x6
      bne  x1, x4, L      # if a != d, goto L
      add  x1, x1, x4     # a = a + d
L:    beq  x5, x6, True   # if i == j, goto True
      sub  x1, x2, x3     # a = b - c, False path
      j    Next           # goto end of branch
True: add  x1, x2, x3     # a = b + c, True path
Next: addi x1, x1, 1      # a++
```

Sometimes we can invert the condition for convenience. *Think carefully about the flow!*
In the above example: `if (a == d)` → bne

30

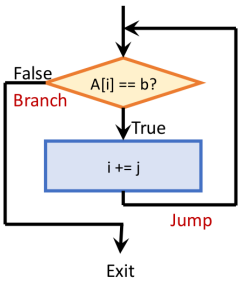
Figure 13: Translation of conditional control flow into branches and jumps, extracted from slide 30. The example shows that assembly frequently uses inverted conditions and explicit jumps to realize the same high-level structured meaning.

A subtle but important point is that assembly often implements an `if` statement by branching on the *opposite* of the high-level condition. This is not a semantic change; it is simply a convenient way to skip over a block when the condition fails. In practice, assembly programming relies heavily on understanding *fall-through*: what happens if the branch is *not* taken.

Loops use both branches and jumps. A conditional test checks whether the loop should continue; an unconditional jump sends control back to the loop header.

Example of while Loop

```
while (A[i] == b)
    i += j;
```




```
# i in x1, j in x2
# b in x3, A in x4
Loop: slli x28, x1, 2      # t0 = i * 4
      add  x29, x4, x28    # t1 = &(A[i])
      lw   x30, 0(x29)     # t2 = A[i]
      bne  x30, x3, End    # if A[i] != b, go to End
      add  x1, x1, x2      # i += j
      j    Loop            # goto next iteration
End:
```

Can also jump backward!

Q: How about do-`while` and `for` loops?

31

Figure 14: A while-loop translation from slide 31. The example shows explicit address calculation for `A[i]`, a branch to exit when the condition fails, and a backward jump to implement iteration.

Once again, the correspondence to high-level programming is exact but indirect. A `while` loop is not a primitive ISA construct; it is a pattern of labels, branches, and jumps. Similarly, `for` and

`do-while` loops are alternative control-flow patterns built from the same underlying mechanisms.

5 ISA Support for Parallelism and Synchronization

5.1 SIMD: single instruction, multiple data

Up to this point, our instructions have operated on one scalar value at a time. Many workloads, however, perform the same operation repeatedly over arrays or vectors. SIMD extends the ISA so that a single instruction applies the same computation to multiple data elements simultaneously.

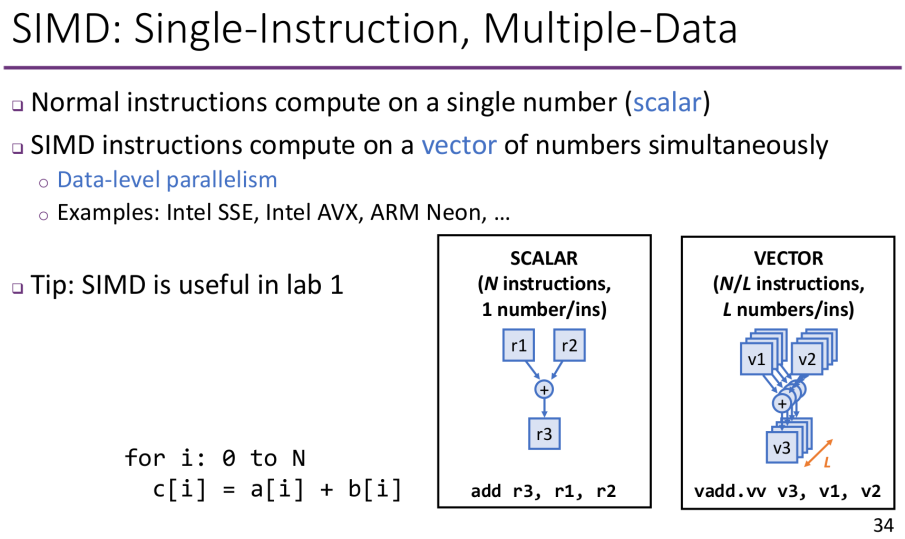


Figure 15: SIMD illustration from slide 34. A scalar instruction processes one value pair, whereas a vector instruction processes multiple lanes in parallel, reducing the number of instructions required for array-style computation.

This is **data-level parallelism**. If a vector register holds L numbers, then a vector addition can potentially replace L scalar additions. If a computation originally required N scalar instructions, the idealized vectorized version may require approximately N/L instructions.

SIMD should not be confused with pipelining or superscalar execution. SIMD means one instruction acts on multiple data elements. Pipelining and superscalar execution mean the hardware overlaps or co-issues multiple instructions. All of these forms of parallelism can coexist.

5.2 Case study: matrix multiplication and vectorization

Matrix multiplication is a natural example because it repeatedly applies multiply-add operations over regular data structures. The scalar form uses three nested loops. For each output element `C[i,j]`, one computes the inner product of a row of `A` and a column of `B`.

Listing 5: Scalar matrix multiplication kernel.

```
void dgemm(size_t n, double* A, double* B, double* C) {
    for (size_t i = 0; i < n; i++) {
        for (size_t j = 0; j < n; j++) {
            double c0 = *(C + i * n + j);
```

```
            for (size_t k = 0; k < n; k++) {
                c0 = c0 + (*(A + i * n + k)) * (*(B + k * n + j));
            }
            *(C + i * n + j) = c0;
        }
    }
}
```

A SIMD version can process multiple values of `j` at once. Conceptually, one loads four adjacent values from `C`, loads four adjacent values from `B`, broadcasts one scalar from `A` into all four lanes, multiplies lane-wise, and accumulates.

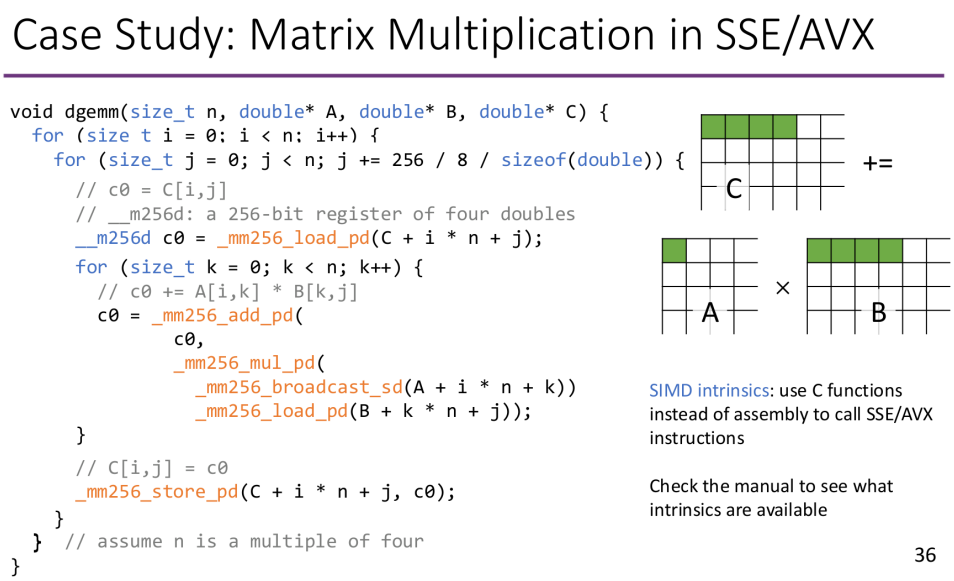


Figure 16: Vectorized matrix multiplication with SIMD intrinsics, extracted from slide 36. The intrinsics correspond to vector instructions such as loads, broadcasts, lane-wise multiply, and lane-wise add.

In C, this is often written using **intrinsics**: compiler-provided functions that map closely to ISA-level SIMD instructions. Intrinsics are useful because they expose vector operations while remaining inside the C type system and compilation pipeline.

The key insight is not the exact AVX syntax, but the execution pattern. Broadcasting one `A[i,k]` value across multiple lanes lets one multiply it against four consecutive values of `B[k,j]` to update four elements of `C[i,j:j+3]` simultaneously. That is exactly the kind of regularity SIMD was designed to exploit.

Review: Multiple Levels of Parallelism

- Not to confuse SIMD with instruction-level parallelism discussed before
- Within a single instruction, process multiple data in parallel
  - SIMD
- Within a single processor, process multiple instructions in parallel
  - Instruction-level parallelism, e.g., pipelining and superscalar
- Among multiple processors
  - Multi-core and GPU
- They can be applied altogether!!
  - E.g., a multi-core CPU uses several superscalar cores, and supports SIMD instructions

37

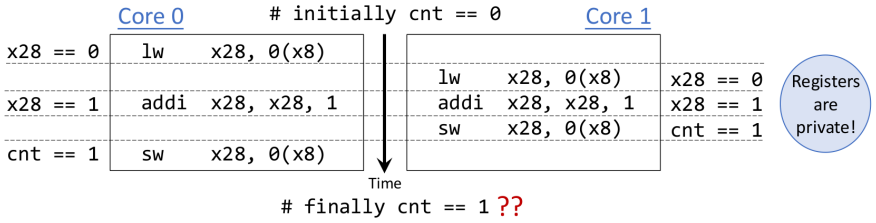
Figure 17: Layering multiple forms of parallelism, from slide 37. SIMD, instruction-level parallelism, and multi-core execution are complementary rather than competing techniques.

5.3 Shared memory, data races, and atomicity

Once multiple cores share memory, correctness problems arise. Consider a shared counter initially equal to 0. Two cores both execute the logical sequence "load the counter, add 1, store the counter." Intuitively, one may expect the final result to be 2. But if both cores load the original value 0 before either stores, then both compute 1 and both store 1. One update is lost.

Data Race and Synchronization

- Data race: if multiple cores modify the same data
- E.g., all attempt to increment a shared counter



- Avoid data races by synchronization of loads/stores to shared data
  - Do read-modify-write atomically on shared data
    - Atomicity: as an undividable entity
  - No other access to the data allowed in between

38

Figure 18: A classic lost-update race from slide 38. Because registers are private to each core but memory is shared, a load–add–store sequence may interleave incorrectly across cores.

This phenomenon is a **data race** in the intuitive sense used in the lecture: multiple cores modify the same shared data without sufficient coordination. The logical problem is that the sequence "read, modify, write" was split into three separate instructions, and another core was allowed to intervene in the middle.

Definition

Atomicity means that an operation behaves as an indivisible unit. Other observers cannot see or interleave with the intermediate states of that operation.

5.4 Atomic instructions and synchronization primitives

To solve this problem, ISAs provide **atomic instructions**. Rather than exposing only ordinary loads and stores, the ISA offers read-modify-write operations that are guaranteed to occur as one indivisible action.

The lecture introduces several common forms:

- **test-and-set**: read a location; if it is zero, set it to one;
- **compare-and-swap**: update a location only if it currently matches an expected value;
- **fetch-and-add**: return the old value while atomically incrementing the memory location.

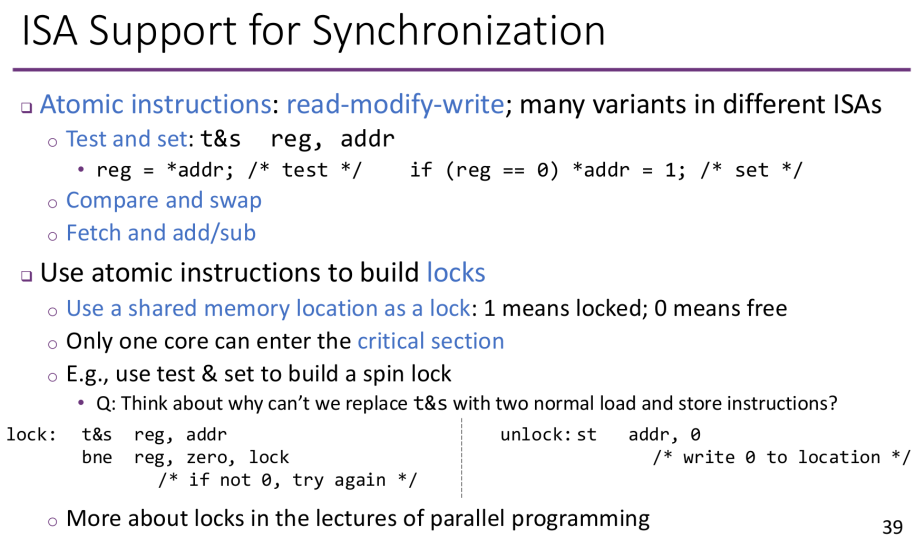


Figure 19: Atomic instructions as the basis for locks, extracted from slide 39. The spin-lock structure repeatedly attempts an atomic test-and-set until the lock becomes available.

A spin lock can be built from test-and-set using a shared memory word where 0 means "unlocked" and 1 means "locked." A core repeatedly performs test-and-set until it observes that the location had previously been 0.

Algorithm 2: Spin Lock Acquisition with Test-and-Set

Input: Address *addr* of a shared lock word, where 0 means free and 1 means locked
Output: Exclusive entry into the critical section

```
while true do
    old ← test_and_set(addr);
    if old = 0 then
        break;
Execute critical section;
Memory[addr] ← 0 ;                                   // unlock
return success;
```

The reason ordinary load and store instructions cannot replace test-and-set should now be clear. They do not prevent interleaving by other cores between the read and the write. Atomic instructions are not "syntactic sugar"; they are architectural support for a correctness property that cannot be guaranteed by naive instruction sequences alone.

The appendix slides also mention variants in the RISC-V atomic extension such as `AMOSWAP` and `AMOADD`. These perform an atomic operation on memory while returning the original value in a register. Architecturally, they are different concrete ways to realize the same general idea: synchronized read-modify-write.

ISA Support for Synchronization

- Atomic instructions: read-modify-write; many variants in different ISAs
  - Test and set: `t&s reg, addr`
    - `reg = *addr; /* test */  if (reg == 0) *addr = 1; /* set */`
  - Compare and swap: `cas cmp_reg, swap_reg, addr`
    - `if (*addr == cmp_reg) swap(*addr, swap_reg);`
  - Fetch and add/sub
    - Read from memory; apply operation on it and store back; return the original value
    - RISC-V, A-extension: fetch and swap/add/and/or/xor/max/min

      ```
      AMOSWAP  rd, rs2, (rs1)
      AMOADD   rd, rs2, (rs1)
      ...
      ```

      Read data from address `rs1`; apply the operation with `rs2` and store back; leave the original value in `rd`

53

Figure 20: Additional examples of synchronization-oriented instructions from slide 53, including compare-and-swap and fetch-and-add style operations.

6 Memory Layout, Stack Management, and Procedure Calls

6.1 The memory layout of a process

When a program is loaded for execution, its address space is organized into regions. The exact layout varies across operating systems and ABIs, but a standard conceptual picture contains the following parts:

- **Text/code**: machine instructions.
- **Static data**: global variables and other data with fixed storage duration.
- **Heap**: dynamically allocated memory requested during execution.
- **Stack**: space used for nested procedure calls.
- **Reserved/kernel regions**: memory not directly under ordinary user-level control.

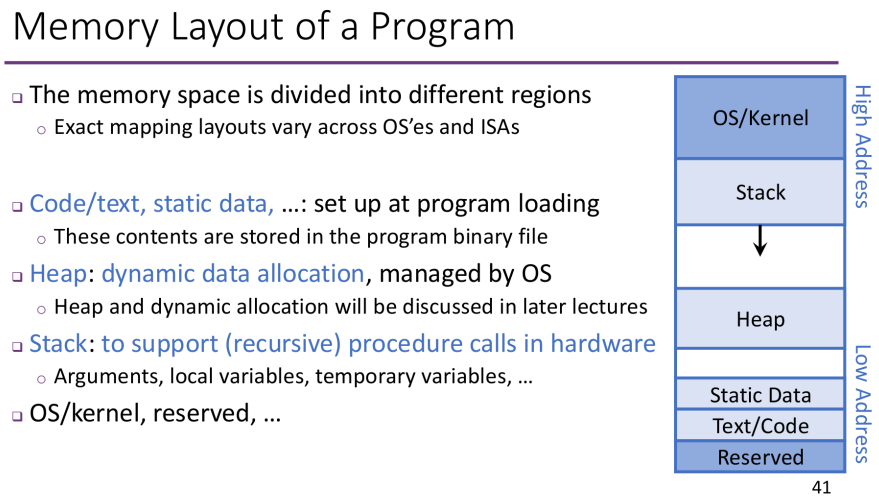


Figure 21: Canonical process memory layout from slide 41. The text and static regions are established when the program is loaded, the heap supports dynamic allocation, and the stack supports procedure calls and local storage.

The code and static data are typically determined at load time from the program binary. The heap is managed dynamically, often through calls such as `malloc` and `free`. The stack is the primary mechanism by which hardware and software cooperate to support procedure calls.

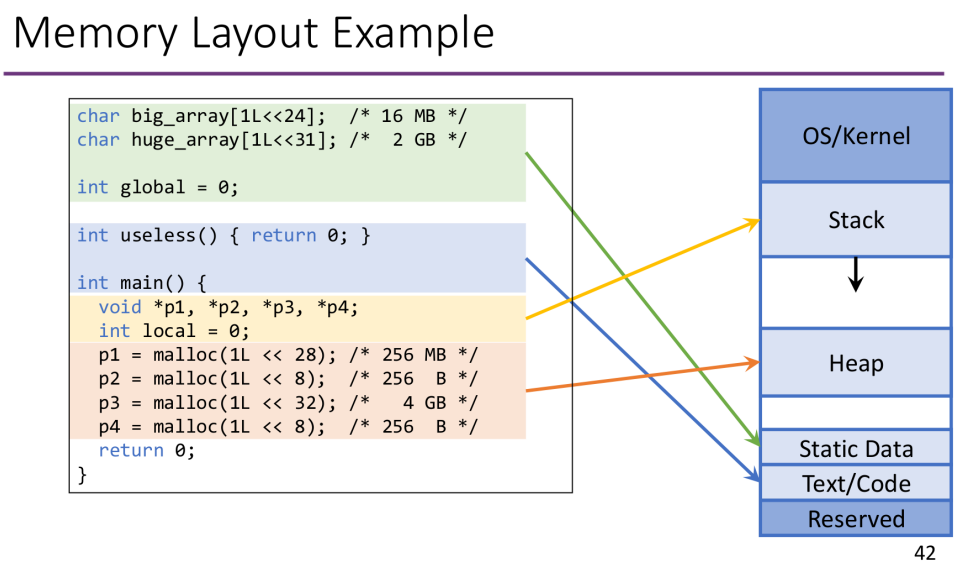


Figure 22: Example program-to-memory mapping from slide 42. Global arrays and global variables occupy static storage, function bodies belong to the text region, local variables occupy stack space, and `malloc` requests are served from the heap.

6.2 Why a stack is needed

Procedure calls require storage for arguments, local variables, temporary values, saved registers, and return information. This storage cannot in general be decided completely at compile time because:

- the number and order of procedure invocations depend on runtime control flow,
- recursive and mutually recursive calls create multiple simultaneously active instances of the same function, and
- the register file is too small to hold all live values in all circumstances.

In principle, one could allocate this storage from the heap. But that would use a general-purpose dynamic allocator for a very common case with a strong and exploitable structure. Function calls are usually **perfectly nested**: a callee returns before its caller returns. This is exactly the discipline a stack is designed to support.

Deep Dive

The stack is a beautiful example of specialized optimization. The heap can support many dynamic-allocation patterns, but it must do so generally and therefore pays general-purpose management costs. Procedure calls, by contrast, almost always follow last-in, first-out behavior. Because the usage pattern is so structured, a much simpler and faster allocator becomes possible: just move a stack pointer.

6.3 Stack frames and nested procedure calls

Each active procedure invocation receives a **stack frame**, also called an activation record. The frame contains the memory storage needed for that invocation. When a function is entered, its frame is pushed onto the stack. When the function returns, its frame is popped.

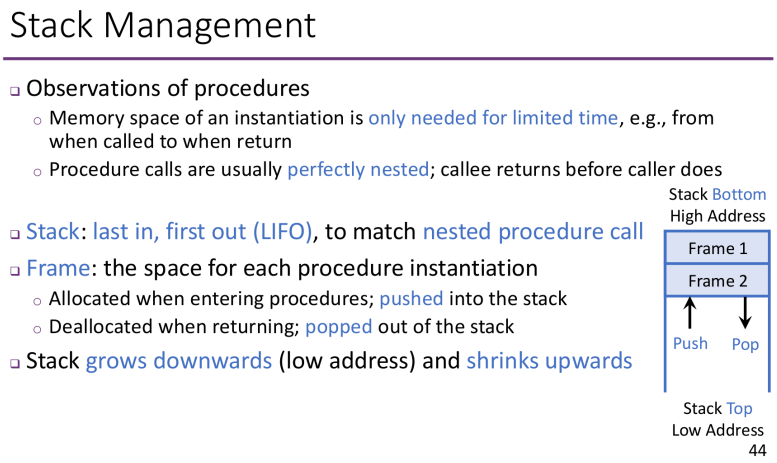


Figure 23: Stack-frame management from slide 44. The stack grows toward lower addresses; each procedure invocation pushes a frame, and returning pops it.

The lecture emphasizes that the stack usually grows downward, from higher addresses to lower addresses. The exact direction is a convention, not a law of nature, but it is widespread. What matters conceptually is the LIFO discipline.

Recursion requires no special exception to this mechanism. Each recursive call is simply another invocation with its own frame.

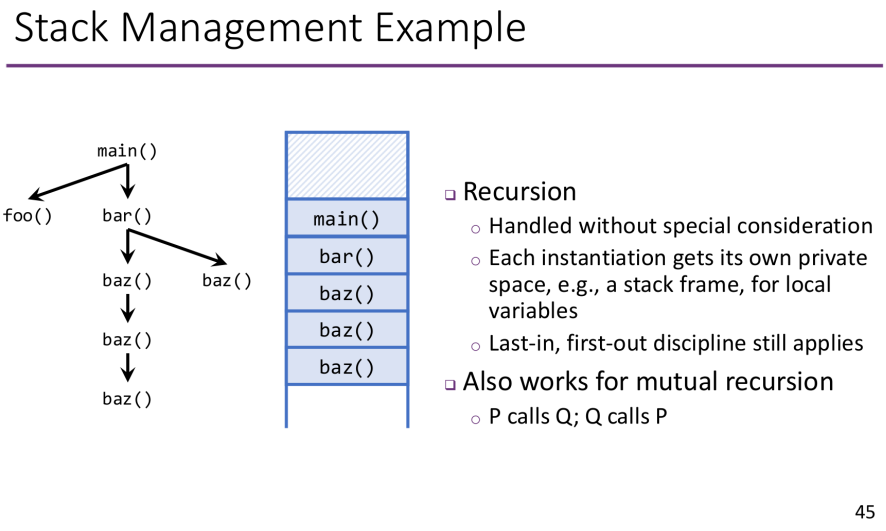


Figure 24: Recursive and nested calls mapped onto stack frames, extracted from slide 45. Even repeated calls to the same function are handled naturally because each invocation receives its own frame.

This is why recursive functions work at all: each activation receives its own private storage for local variables and temporaries. If all calls to a function shared one memory area, recursion

would destroy correctness immediately.

6.4 Register interference across procedure calls

Memory frames solve the problem of local storage, but another problem remains: registers are global hardware resources shared across calls. Suppose function `A()` is using certain registers and then calls `B()`. If `B()` also uses some of those registers, it may overwrite values that `A()` still needs after the call returns.

The standard solution is to divide responsibility through a calling convention. Some registers are classified as **caller-saved**, meaning the caller must preserve them if it still needs their values after the call. Others are **callee-saved**, meaning the callee must save and restore them if it chooses to use them. The important systems idea is that this is a *contract*. Independently written code can interoperate safely because both sides follow the same convention.

6.5 Procedure calls as control transfer plus state management

A function call is more than just a jump. It must also remember where to return. In the lecture's simplified RISC-V model, the instruction

```
jal L
```

performs a *jump and link*: it jumps to label `L` and simultaneously saves the return address into the register `ra`, which is another name for `x1`. Returning is performed by jumping to the address stored in `ra`, for example using

```
jr ra
```

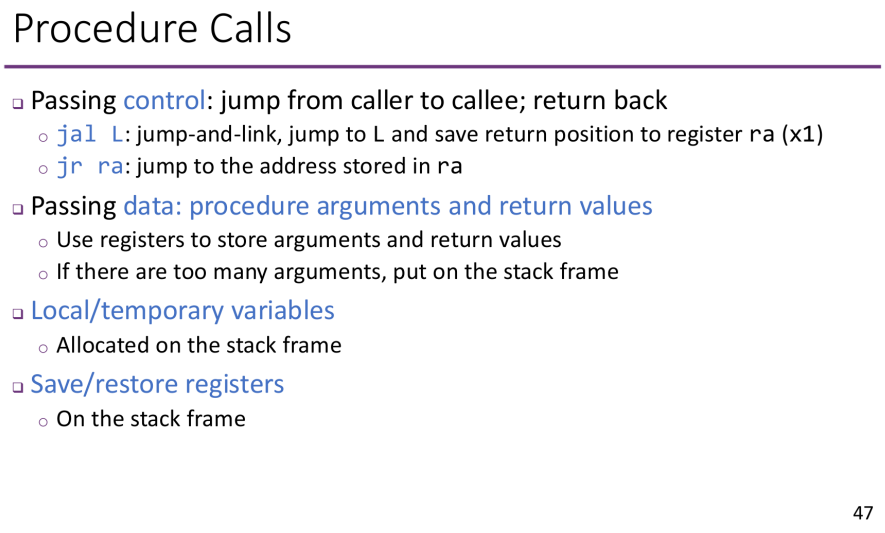


Figure 25: Procedure-call mechanics from slide 47. A call transfers control to the callee while storing the return position; arguments, return values, local storage, and save/restore data are organized through registers and the stack frame.

In a real calling convention, the stack frame often contains some combination of the following:

- saved return address,
- saved callee-saved registers,
- spilled temporaries,
- local variables that do not fit in registers,
- outgoing arguments for deeper calls.

Arguments and return values are usually passed in a small number of designated registers when possible; excess arguments spill onto the stack. This illustrates a recurring systems theme: registers are used first for speed, and memory is used as overflow storage when capacity is insufficient.

6.6 Programming implications and common pitfalls

The memory-layout and stack model has several practical consequences that every computer science student should understand.

1. Local variables have limited lifetime. A pointer to a local stack variable becomes invalid after the function returns. Using it afterward is a classic form of undefined behavior.

2. Deep recursion consumes stack space. If recursion is too deep, the stack may overflow. This is one reason iterative algorithms are sometimes preferred even when recursion is conceptually elegant.

3. Large local arrays are dangerous. Allocating a very large object on the stack may exhaust stack space quickly. Such objects are often better placed in static storage or allocated on the heap.

4. Corrupting stack metadata is serious. Because the stack often contains saved control information such as return addresses, erroneous writes into stack memory can affect not only data but control flow as well. This is one reason buffer overflows are such important systems-security bugs.

Seen this way, the stack is not just a compiler convenience. It is a central data structure in the execution model of real programs.

7 Summary

This lecture has traced a complete path from abstract computation to concrete program execution.

- A computer system can be modeled as a mechanism that receives input, computes over stored state, and produces output.
- The stored-program model separates the processor from memory and allows both code and data to live in memory.
- Registers exist because heavily used values need a storage location much faster than main memory.

- Instruction execution is naturally described in stages: fetch, decode, execute, memory access, and write-back.
- Performance improvements such as pipelining, superscalar execution, and multi-core organization exploit different forms of parallelism while preserving ISA-level correctness.
- RISC-style ISAs such as RV32I keep instructions simple and regular, making the software–hardware interface easier to understand.
- Assembly code reveals how arithmetic, memory access, and control flow are expressed using explicit registers, load/store operations, branches, and jumps.
- SIMD instructions exploit data-level parallelism, while atomic instructions preserve correctness when multiple cores interact through shared memory.
- The process memory layout organizes code, static data, heap storage, and the stack into distinct regions with different roles.
- The stack is the natural mechanism for supporting nested procedure calls because function activations usually follow a last-in, first-out lifetime pattern.

The most important unifying idea is that abstraction does not eliminate mechanism; it organizes it. High-level languages, ISAs, microarchitectures, and runtime memory layouts are different layers of one coherent system. To understand computer systems well, one must continually connect those layers rather than studying them in isolation.

8 Quizzes & Exercises

8.1 Concept checks drawn from the lecture

1. What is the effect of the instruction `xor x1, x1, x1`? Why is it useful even though it may look redundant at first glance?
2. For the expression `a = (b ^ c) + d - e`, what is the minimum number of temporary registers required if the old value of `a` does not need to be preserved?
3. What are the advantages of displacement addressing for arrays, structures, and stack-frame access?
4. How would you realize a `do-while` loop using only labels, branches, and jumps?
5. Why can a spin lock not be implemented correctly by simply replacing test-and-set with an ordinary load followed later by an ordinary store?

8.2 Short-answer quiz

Answer each question in two or three sentences.

1. What are the primary components of the Von Neumann architecture, and how do they interact?

2. How does the Harvard architecture differ from the Von Neumann architecture?
3. Explain the role of the Program Counter (PC) in a processor.
4. Why are registers used as an optimization despite their limited number compared to memory?
5. Describe the five classic stages of instruction execution in a processor.
6. What is the difference between pipelining and superscalar execution?
7. Define Instruction Set Architecture (ISA) and explain its role as an interface.
8. Compare the philosophies of RISC and CISC architectures.
9. What is displacement addressing in the context of RV32I memory access?
10. What characterizes specialized hardware accelerators such as the Google TPU?

8.3 Essay and discussion prompts

Develop comprehensive responses to the following prompts.

1. **The evolution of parallelism.** Discuss the transition from simple serial processors to pipelined, superscalar, multi-core, GPU, and accelerator-oriented designs. In your answer, distinguish clearly between instruction-level, data-level, and thread-level parallelism.
2. **The abstraction of ISA.** Evaluate the importance of ISA as a stable abstraction layer. How does the separation of architecture from microarchitecture benefit both software developers and hardware designers?
3. **Registers versus memory.** Analyze the relationship between fast local storage and larger but slower memory. Why is the "smaller is faster" principle central to processor design, and how do `lw` and `sw` mediate between the two levels?
4. **Why RISC-V is pedagogically effective.** Explain why a clean and regular ISA is especially valuable in an introductory computer-systems course. Compare how compound work is expressed in a RISC environment versus a more CISC-like environment.
5. **Program translation and execution.** Trace the lifecycle of a program from a high-level source file to execution in memory. Explain the roles of the compiler, assembler, linker, and loader.

8.4 Additional practice exercises

1. Translate the following assembly fragment into equivalent C code, assuming `x1` holds `i`, `x2` holds `j`, `x3` holds `b`, and `x4` holds the base address of `A`:

```
Loop:
    slli x28, x1, 2
    add  x29, x4, x28
    lw   x30, 0(x29)
    bne  x30, x3, End
```

```
6       add  x1, x1, x2
7       j    Loop
8   End:
```

2. Suppose a processor has a five-stage pipeline. Explain why a branch instruction may reduce the ideal speedup of pipelining even when no arithmetic dependencies are present.

3. A function allocates a very large array as a local variable. Discuss why this may be problematic and compare at least two alternative allocation strategies.

4. Consider two cores incrementing a shared counter using ordinary load, add, and store instructions. Draw one interleaving that leads to a lost update.

5. Explain how caller-saved and callee-saved registers together form a contract that allows separately compiled functions to interoperate correctly.